

Scientific Programming 2 - Practice Exam

This practice exam has the same form as the real exam: three assignments, to be made in two hours.

- Make the practice exam in the online exam editor, linked from the course website (sp.proglab.nl).
- Put the solutions of all three assignments in the single file `sp2_practice_exam.py`. It is already open in the editor.
- The Checkpy button checks your solutions.
- You may use the course website as a resource, and a printed version of the cheatsheet. The same goes for the real exam: there too you may use the course website (the cheatsheet is on it as well) and a printed cheatsheet.
- Do not use external modules such as `numpy`, `csv`, or others unless the assignment specifically says you can.
- You are evaluated solely on the correctness of your solutions. Code design, comments, and style are not important.

Assignment 1: Translations

Imagine we have two Python dictionaries for translations, for instance an English-French and a French-Spanish dictionary. Write a function `combine_dicts(dict1, dict2)` that combines them into an English-Spanish dictionary. Words whose translation cannot be found in the second dictionary are left out.

Example Usage

```
english_to_french = {'banana': 'banane', 'apple': 'pomme', 'almond': 'amande',  
                    'cat': 'chat', 'fine': 'amande'}  
french_to_spanish = {'pomme': 'manzana', 'car': 'coche', 'banane': 'plátano',  
                    'amande': 'almendra'}
```

```
english_to_spanish = combine_dicts(english_to_french, french_to_spanish)  
print(english_to_spanish)
```

Expected Output

```
{'banana': 'plátano', 'apple': 'manzana', 'almond': 'almendra', 'fine': 'almendra'}
```

Assignment 2: Duplicate Values

Write a function `find_pairs_with_duplicate_values(mapping)` that takes a dictionary and finds all pairs of keys that have the same value. It returns a set of tuples, one tuple for each pair, with the two keys of the pair in alphabetical order.

Example Usage

```
mapping = {"noodle": "long thin pasta",  
          "Ferrari F40": "expensive supercar",  
          "spaghetti": "long thin pasta",  
          "Casio F-91W": "popular watch",  
          "McLaren F1": "expensive supercar",  
          "vermicelli": "long thin pasta",  
          "novel": "something to read"}  
  
print(find_pairs_with_duplicate_values(mapping))
```

Expected Output

```
{('noodle', 'spaghetti'), ('Ferrari F40', 'McLaren F1'),  
 ('noodle', 'vermicelli'), ('spaghetti', 'vermicelli')}
```

Note: a set has no fixed order, so the pairs may be printed in a different order.

Assignment 3: Export

In this assignment you implement two classes, `Country` and `Continent`, that model the export of countries and continents.

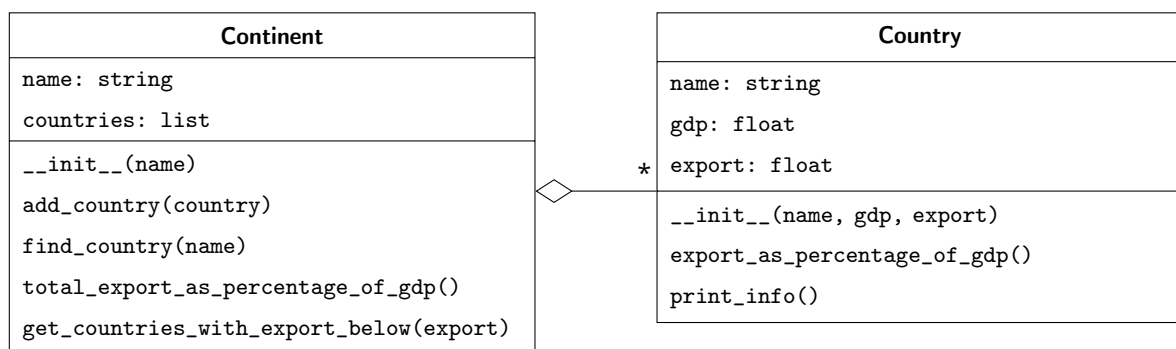
A `Country` has three attributes:

- `name`: the name of the country;
- `gdp`: the country's gross domestic product (GDP) in millions of dollars;
- `export`: the country's total export in millions of dollars.

A `Continent` has two attributes:

- `name`: the name of the continent;
- `countries`: a list of the `Country` objects that belong to the continent. It starts empty.

The following UML diagram describes the two classes and their relation:



`Country` has these methods:

- `__init__(name, gdp, export)`: create a new country with the given name, GDP and export.
- `export_as_percentage_of_gdp()`: return the country's export as a percentage of its GDP.
- `print_info()`: print the information about the country, in the format shown in the example.

`Continent` has these methods:

- `__init__(name)`: create a new continent with the given name and no countries yet.
- `add_country(country)`: add a `Country` object to the list of countries.
- `find_country(name)`: return the `Country` object with the given name, or `None` if there is none.
- `total_export_as_percentage_of_gdp()`: return the total export of all countries of the continent, as a percentage of their total GDP.
- `get_countries_with_export_below(export)`: return a list of the `Country` objects whose export is below `export`.

Example Usage

The numbers are from 2021, in millions of dollars (source: worldbank.org).

```
europe = Continent("Europe")
europe.add_country(Country("Italy", 2107702.84, 688582.42))
europe.add_country(Country("France", 2957879.76, 871058.67))
europe.add_country(Country("U.K.", 3131377.76, 875196.92))
europe.add_country(Country("Germany", 4259934.91, 2003471.01))
europe.add_country(Country("Spain", 1427380.68, 498633.85))

asia = Continent("Asia")
asia.add_country(Country("Kazakhstan", 197112.26, 66124.96))
asia.add_country(Country("India", 3176295.07, 679680.59))
asia.add_country(Country("Mongolia", 15286.44, 8950.04))
asia.add_country(Country("China", 17734062.65, 3553509.24))
asia.add_country(Country("Indonesia", 1186092.99, 255731.27))

kazakhstan = asia.find_country("Kazakhstan")
kazakhstan.print_info()

print("European countries with export below $700,000M:")
countries = europe.get_countries_with_export_below(700000)
for country in countries:
    country.print_info()

print(f"Total export as % of GDP of Asia: {asia.total_export_as_percentage_of_gdp():.2f}")
```

Expected Output

```
Country: Kazakhstan; gdp: 197112.26; export: 66124.96; export as % of gdp: 33.54685294562601
European countries with export below $700,000M:
Country: Italy; gdp: 2107702.84; export: 688582.42; export as % of gdp: 32.66980557847519
Country: Spain; gdp: 1427380.68; export: 498633.85; export as % of gdp: 34.933487400151726
Total export as % of GDP of Asia: 20.46
```